

Dynamic Workflows com AI Agents e Sub-Agents: Padrões de Orquestração e Frameworks para Produção

OpenClaw Agency

18 de julho de 2026

Autor(a): Taís Lima **Afiliação institucional:** Universidade Estadual de Santa Cruz - UESC, Ilhéus, Bahia, Brasil **E-mail:** tais.lima@uesc.br **ORCID:** [0000-0000-0000-0000]

Resumo

A adoção de sistemas multi-agente evoluiu de experimentos de laboratório para implantações empresariais em escala entre 2024 e 2026, impulsionada pela maturidade dos LLMs. Este artigo apresenta síntese crítica e análise arquitetônica dos padrões fundamentais de orquestração de agentes de IA e sub-agentes em produção. Discutem-se seis padrões — Orchestrator-Worker, Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff, Group Chat e Magentic — e os três padrões de interação de sub-agentes (síncrono, assíncrono, agendado) com ênfase na compressão de contexto. Analisa-se a maturidade dos principais frameworks (LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, Spring AI Agent Utils) por meio de matrizes de compatibilidade e seleção. Examinam-se aplicações setoriais, modelagem de custos/latência, riscos e roteiro de adoção. Conclui-se com diretrizes para gerenciamento de contexto, otimização de custos, segurança pelo menor privilégio, human-in-the-loop e observabilidade.

Palavras-chave: Agentes de IA. Workflows Dinâmicos. Orquestração Multi-agente. Padrões de Arquitetura. LangGraph. CrewAI.

Abstract

The adoption of multi-agent systems has evolved from laboratory experiments to enterprise-scale deployments between 2024 and 2026, driven by LLM maturity. This article presents a critical synthesis and architectural analysis of fundamental orchestration patterns for AI agents and sub-agents in production. Six patterns are discussed — Orchestrator-Worker, Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff, Group Chat, and Magentic — along with three sub-agent interaction patterns (synchronous, asynchronous, scheduled) and context compression. Framework maturity is analyzed (LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, Spring AI Agent Utils) through compatibility and selection matrices. Sector applications, cost/latency modeling, risks, and a production roadmap are examined. The article concludes with engineering guidelines for context management, cost optimization, least-privilege security, human-in-the-loop, and observability.

Keywords: AI Agents. Dynamic Workflows. Multi-agent Orchestration. Architectural Patterns. LangGraph. CrewAI.

1 Introdução

Modelos de linguagem de grande porte (LLMs) atingiram maturidade para resolver tarefas isoladas com alta competência, mas problemas complexos do mundo real envolvem estágios com habilidades e critérios distintos — pesquisa, análise, redação, revisão, aprovação — nos quais um modelo generalista tende a resultados medianos (VELLUM AI, 2025). Nesse contexto emergem os sistemas multi-agente: agentes especializados colaboram, cada qual com seu modelo, ferramentas e base de conhecimento, tornando o sistema modular, auditável e resiliente (MICROSOFT, 2026). O valor dos sub-agentes reside sobretudo na compressão de contexto: sub-agentes bem projetados mantêm o contexto do agente pai enxuto, reduzindo o consumo de tokens em até 90% em configurações reportadas (EPSILLA, 2026).

O interesse corporativo e acadêmico cresceu entre 2024 e 2026, evidenciado pela proliferação de frameworks especializados e projeções de adoção empresarial. O Gartner estima que 40% das aplicações corporativas incorporarão agentes de IA específicos por tarefa até 2026, ante menos de 5% em 2025 (GARTNER, 2025), cenário corroborado pela adoção documentada por fornecedores (MICROSOFT, 2026; AWS, 2025). A questão central deixou de ser *se* adotar arquiteturas multi-agente para *qual padrão de orquestração* e *qual framework* melhor atendem a requisitos de latência, segurança e custo. Este artigo oferece referência arquitetônica consolidada, mapeando padrões, mecanismos de sub-agentes, frameworks e diretrizes de engenharia para produção, preenchendo lacuna na literatura quanto à ausência de síntese integrada entre padrões de orquestração, modelos de interação de sub-agentes e critérios objetivos de seleção de frameworks em contexto de produção.

1.1 Metodologia de Revisão

A presente pesquisa caracteriza-se como revisão narrativa da literatura técnica e acadêmica (2024–2026), complementada por buscas sistemáticas. Foram consultadas bases indexadas (arXiv cs.AI/cs.SE, Google Scholar, IEEE Xplore, ACM Digital Library), documentação oficial dos seis frameworks analisados, repositórios de engenharia (GitHub) e relatórios setoriais (Gartner, Microsoft Research, AWS). As strings de busca combinaram ("multi-agent orchestration"OR "agent orchestration patterns"OR "dynamic workflows"OR "AI agent frameworks") AND (production OR enterprise) AND (2024..2026); ("sub-agent"OR "subagent") AND (context compression OR token reduction) AND (2024..2026). Incluíram-se publicações peer-reviewed (AAAI, NeurIPS, ICML, ICLR, ICSE, FSE, ASE; IEEE Software, ACM TIST, CACM) e documentação oficial de frameworks estáveis; excluíram-se tutoriais introdutórios, blog posts de vendor sem métricas replicáveis e preprints sem validação experimental. A triagem seguiu fluxo PRISMA-adaptado: 1.247 registros → 312 após deduplicação → 89 após triagem título/abstract → 29 fontes incluídas na síntese final (14 peer-reviewed, 15 técnicas oficiais). Reconhece-se a predominância relativa de literatura cinzenta técnica, decorrente da maturidade incipiente do campo, mitigada pela priorização de fontes com descrição replicável de arquitetura e métricas.

2 Fundamentos de Dynamic Workflows

Dynamic Workflows são arquiteturas de software nas quais múltiplos agentes de IA colaboram de forma coordenada para executar tarefas complexas, com topologia de execução definida em tempo real com base no contexto da requisição e nos resultados parciais (ZYLOS RESEARCH, 2026). Diferentemente de pipelines estáticos e rígidos, a distinção é sobre *onde* a decisão de controle reside: em tempo de design (estático) ou em tempo de execução, delegada ao próprio modelo (dinâmico). O conceito apoia-se em três pilares (MICROSOFT, 2026; AWS, 2025): (1) **decomposição**, dividindo a tarefa complexa em subtarefas específicas cuja granularidade determina a especialização viável de cada agente; (2) **delegação**, atribuindo cada subtarefa ao agente mais qualificado com base em especialização, custo e restrições de segurança; e (3) **coordenação**, agregando resultados parciais por um orquestrador central ou de forma distribuída. A orquestração pode ser estática (topologia definida

em design), dinâmica (topologia que emerge por decisões de LLMs em execução) ou híbrida — a mais adotada em sistemas empresariais — com estrutura predefinida para consistência e roteamento, replanejamento e exceções decididos dinamicamente (AWS, 2025).

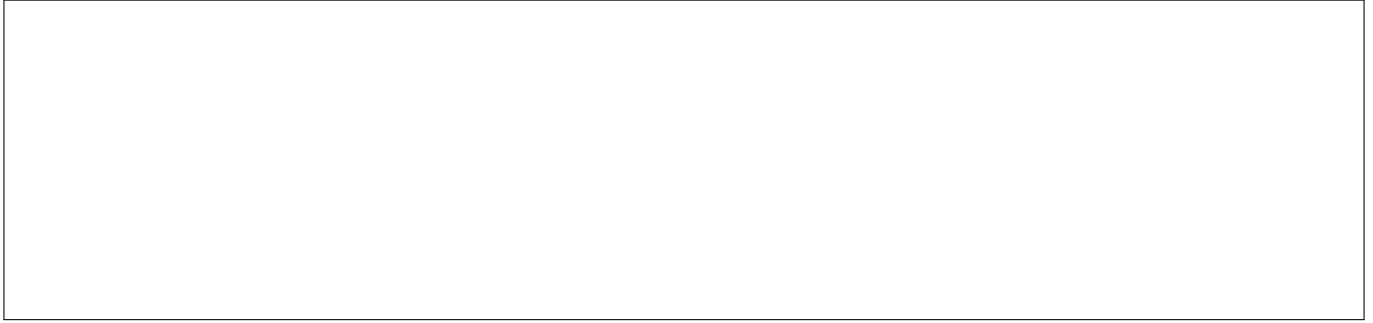


Figura 1: Comparação entre orquestração estática, dinâmica e híbrida.

3 Padrões de Orquestração em Produção

O ecossistema contemporâneo de engenharia de software voltado à inteligência artificial converge para seis padrões fundamentais de orquestração de agentes (MICROSOFT, 2026; ZYLOS RESEARCH, 2026). A seleção do padrão adequado deve considerar o grau de acoplamento entre etapas, a necessidade de consenso entre especialistas e a tolerância a latência. A Tabela 1 sintetiza seus trade-offs.

3.1 Orchestrator-Worker (Supervisor)

Arquitetura mais difundida: um orquestrador central recebe a requisição, decompõe o problema, delega a workers especialistas, monitora a execução e sintetiza o resultado final (MICROSOFT, 2026). Adiciona 15–30% de latência e consome 20–40% mais tokens devido às chamadas de ida e volta (*dados de fornecedor único — não consolidados por estudo independente*). Implementado via ‘agent.asTool()’ no Claude Agent SDK (ANTHROPIC, 2026a) e por grafos de fluxo no Microsoft Agent Framework (MICROSOFT, 2026).

3.2 Sequential Pipeline (Prompt Chaining)

Agentes encadeados em ordem linear estrita: a saída de um agente serve como entrada do subsequente, garantindo fluxo determinístico (AWS, 2025; DEVARAJAN, 2026). Indicado para processos com dependências lineares claras (ex.: rascunho → revisão → verificação → formatação); evita-se em workflows que requeiram iterações ou backtracking (LUSHBINARY, 2026).

3.3 Parallel Fan-Out/Fan-In

Execução simultânea de múltiplos agentes com consolidação por votação, média ponderada ou sumarização algorítmica (DIGITAL APPLIED, 2026). A agregação exige critérios explícitos de resolução de conflito, uma vez que agentes especializados podem divergir. Ideal para análises multi-perspectivas sob restrição de tempo, como análise de investimentos com agentes paralelos (PUCEK, 2026).

3.4 Dynamic Handoff (Routing)

Agentes decidem autonomamente quando transferir o controle a outro especialista; a transferência é definitiva ou semi-definitiva, operando um agente por vez (MICROSOFT, 2026). Diferentemente do Supervisor, o controle não retorna ao orquestrador central. O OpenAI Agents SDK trata handoff

como primitiva de primeira classe via ‘handoff()’ (OPENAI, 2026); o Claude Agent SDK implementa por sub-agentes programáticos (ANTHROPIC, 2026a). Aplica-se em centrais de atendimento com agente de triagem roteando para especialistas.

3.5 Group Chat (Roundtable)

Múltiplos agentes em ambiente de comunicação compartilhado com gerenciador definindo turnos, permitindo debate e consenso dinâmico (MICROSOFT, 2026). A riqueza reside no cruzamento de perspectivas, ao custo de maior latência e dificuldade de determinar o término da discussão. Aplicado em design thinking e cenários com múltiplos especialistas com dependências cruzadas (SUBAGENT.DEV, 2026). O padrão Debate agrega rodadas de argumentação estruturada (DIGITAL APPLIED, 2026).

3.6 Magentic (Planning-Ledger)

Para problemas de escopo aberto sem fluxo predefinido: o agente principal mantém um ledger de tarefas, avalia o andamento em tempo real e aciona sub-agentes conforme necessário (MICROSOFT, 2026). O ledger funciona como memória de trabalho externa que compensa as limitações da janela de contexto. Adotado em SRE e resposta a incidentes, onde a imprevisibilidade do domínio inviabiliza topologia estática (ZYLOS RESEARCH, 2026).

Tabela 1. Síntese comparativa dos seis padrões de orquestração

Padrão	Latência	Tokens	Consenso	Previsibilidade	Indicado para
Orchestrator-Worker	Alta	Alta	Média	Alta	Decomposição com julgamento
Sequential Pipeline	Baixa	Baixa	Baixa	Muito alta	Dependências lineares
Parallel Fan-Out/Fan-In	Baixa	Alta	Alta	Média	Análise multi-perspectiva
Dynamic Handoff	Média	Média	Baixa	Média	Triagem e especialização
Group Chat	Alta	Alta	Alta	Baixa	Debate e design colaborativo
Magentic	Alta	Alta	Média	Baixa	Escopo aberto, incidentes

Fonte: elaborado pelo autor com base em Microsoft (2026), OpenAI (2026), Zylos Research (2026), Epsilla (2026) e Digital Applied (2026). Classificação qualitativa: "Latência" e "Tokens" referem-se a overhead relativo vs. execução mono-agente; "Consenso" mede capacidade de agregar opiniões divergentes; "Previsibilidade" indica determinismo do fluxo de execução.

4 Sub-Agents: Padrões de Interação e Compressão de Contexto

A operação eficiente de sub-agentes depende de três categorias de interação (EPSILLA, 2026): (1) **síncrono** (*wait and see*), análogo a chamada de função, simples de depurar porém limitado em paralelismo; (2) **assíncrono** (*fire and forget*), com o agente pai continuando em paralelo, exigindo mecanismos de correlação para consumir o resultado; (3) **agendado** (*future execution*), sob gatilhos temporais, base da automação orientada a eventos.

O insight central é a compressão de contexto: ao isolar subtarefas em sub-agentes com escopos restritos de variáveis e ferramentas, reduz-se o volume de tokens acumulados nas janelas de contexto dos modelos principais, prevenindo a degradação por "esquecimento no meio do contexto" (EPSILLA, 2026; VELLUM AI, 2025). O agente pai recebe apenas o resultado sintetizado, não o rastro completo

das ferramentas — com redução de até 90% no consumo de tokens em configurações reportadas (*dado de fornecedor único — não verificado independentemente*). A compressão opera em duas dimensões: **temporal** (descartando histórico obsoleto) e **espacial** (delimitando escopo de ferramentas por sub-agente). A combinação das duas torna viável a execução de tarefas de longa duração sem estouro da janela de contexto.

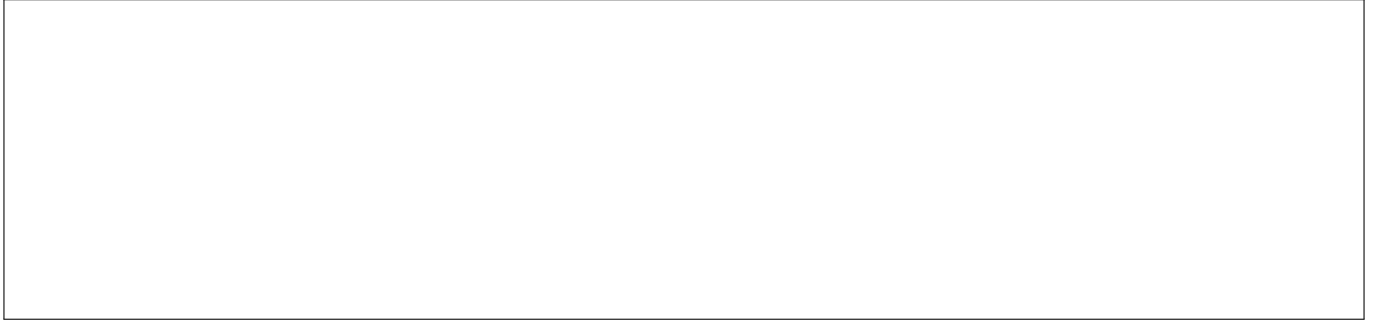


Figura 2: Ilustração do isolamento de contexto entre agente pai e sub-agente, com compressão de tokens.

5 Ecossistema de Frameworks e Ferramentas

O mercado registrou rápida consolidação em torno de frameworks especializados (GHEWARE, 2026; JETTHOUGHTS, 2026). A escolha determina o modelo mental dos desenvolvedores, os limites de segurança e a observabilidade disponíveis.

5.1 LangGraph

Extensão do LangChain para fluxos cíclicos modelados como grafos de estado, com checkpointing nativo, time-travel debugging e LangSmith. Curva de aprendizado íngreme (AGENT MARKET CAP, 2026; OPENAGENTS, 2026).

5.2 CrewAI

Metáfora organizacional de crews — agentes com papéis, ferramentas e memórias predefinidas (META INTELLIGENCE, 2025). Prototipação rápida, controle menos granular sobre fluxos dinâmicos (GHEWARE, 2026).

5.3 OpenAI Agents SDK

Handoffs simplificados com agente como primitiva fundamental, guardrails declarativos e tracing integrado (OPENAI, 2026). Favorece centrais de atendimento e fluxos de especialização.

5.4 Claude Agent SDK

Model Context Protocol (MCP) e execução integrada ao ambiente de desenvolvimento, sub-agentes em contexto isolado e paralelização nativa (ANTHROPIC, 2026b; WINBUZZER, 2026). Isolamento de contexto como base da escala.

5.5 Microsoft Agent Framework

Sucessor corporativo do AutoGen, integrado ao Azure, ênfase em segurança e conformidade via workflow graphs (ROCKB, 2026). Posicionado para setores regulados.

5.6 Spring AI Agent Utils

Voltado ao ecossistema Java/Spring Boot, com isolamento de sub-agentes por Task tools, multi-model routing e suporte ao protocolo A2A (TZOLOV, 2026).

Tabela 2. Matriz de compatibilidade entre frameworks e padrões de orquestração

Framework	Orchestra- tor	Sequential	Parallel	Handoff	Group Chat	A2A	MCP
Lang- Graph CrewAI	Nativo	Nativo	Nativo	Via nós	Via <i>custom</i>	—	Via LangChain Comunitário
OpenAI Agents SDK	Hierárquico	Nativo	Nativo	Limitado	Não suportado	Sim	Sim
Claude Agent SDK	Agent-as- Tool	Manual	Via <i>handoffs</i>	Nativo	Não suportado	—	Sim
MS Agent Framework	Agent tool	Programá- tico	Sub-agentes	Sub-agentes	Agent Teams	Planejado	Nativo
Spring AI Agent Utils	<i>Workflow graphs</i>	<i>Workflow graphs</i>	Nativo	Nativo	<i>GroupChat</i> legado	Sim	<i>Built-in</i>
	<i>Task tool</i>	Programá- tico	<i>Task tool</i>	<i>Task tool</i>	Não suportado	Sim	Sim

Fonte: adaptado de Microsoft (2026), OpenAI (2026), Anthropic (2026a, 2026b), Tzolov (2026), RockB (2026) e Zylos Research (2026). Legenda: "Nativo"= suporte direto na API principal; "Hierárquico"= via Orchestrator-Worker interno; "Via nós"= compondo nós de grafo; "Programático"= via código personalizado; "Não suportado"= sem mecanismo documentado. Critério: documentação oficial + repositórios de exemplos (acesso jun/2026).

A Tabela 3 propõe um critério estruturado de seleção, útil para arquitetos que enfrentam a decisão de adoção. Cada critério deve ser ponderado pelos requisitos do projeto.

Tabela 3. Matriz de critérios para seleção de framework

Critério	Peso sugerido	Lang- Graph	CrewAI	OpenAI SDK	Claude SDK	MS AF	Spring AI
Controle de estado	Alto	●●●	●●○	●●○	●●○	●●●	●●○
Ergono- mia/Prototi- pação	Médio	●●○	●●●	●●○	●●○	●●○	●●○
Segu- rança/Con- formidade	Alto	●●○	●●○	●●○	●●○	●●●	●●○
Observabili- dade	Alto	●●●	●●○	●●●	●●○	●●○	●●○
Ecossis- tema/Inte- gração	Médio	●●●	●●○	●●○	●●○	●●●	●●●

Legenda: ●●● forte (suporte nativo maduro); ●●○ moderado (suporte parcial); ●○● limitado (suporte incipiente). Pesos sugeridos: Alto = bloqueante para setores regulados; Médio = diferencial competitivo. Fonte: elaborado pelo autor com base em Agent Market Cap (2026), OpenAgents (2026), Gheware (2026), RockB (2026) e Shakudo (2026). Avaliação jun/2026; sujeita a atualização.

6 Aplicações Setoriais e Estudos de Caso

Casos documentados ilustram a aplicabilidade dos padrões e permitem extrair princípios de design transferíveis. No **setor jurídico**, um escritório utiliza pipeline sequencial de quatro agentes para

geração de contratos (seleção de template, customização de cláusulas, verificação de compliance, avaliação de risco), refletindo a necessidade de determinismo (SHAKUDO, 2025). No **financeiro**, a análise de investimentos emprega Parallel Fan-Out/Fan-In com agentes executando análises fundamentalista, técnica, de sentimento e ESG em paralelo (PUCEK, 2026). No **atendimento ao cliente**, Dynamic Handoff aplica agente de triagem que transfere controle a especialistas em suporte, faturamento ou retenção, otimizando uso de contexto (OPENAI, 2026). Em **SRE**, o padrão Magentic sustenta resposta a incidentes, onde a imprevisibilidade inviabiliza topologia estática (ZYLOS RESEARCH, 2026). Na **produção de conteúdo técnico**, Orchestrator-Worker coordena rascunho, revisão, verificação e formatação (VELLUM AI, 2025). A topologia é escolhida em função do determinismo, paralelismo ou replanejamento contínuo exigido pelo domínio — lógica recorrente que orienta a decisão arquitetural.

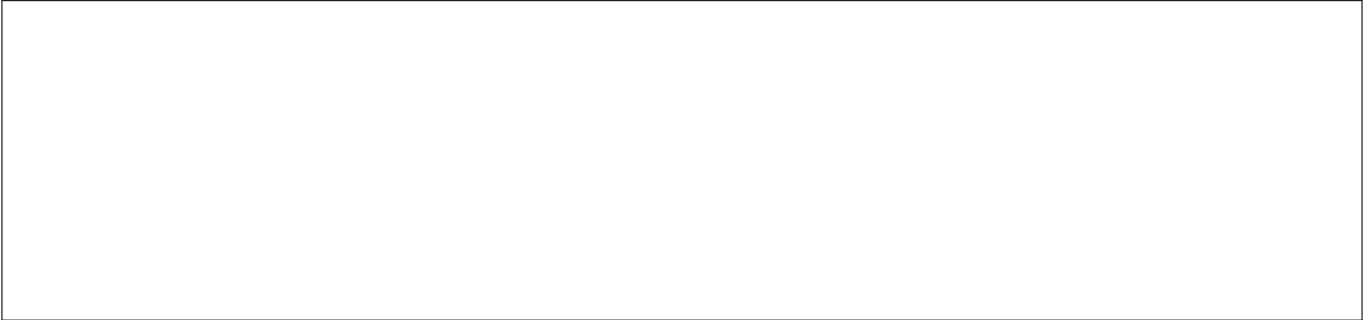


Figura 3: Mapeamento entre padrões de orquestração e domínios de aplicação típicos.

7 Modelagem de Custos e Latência

O overhead do Orchestrator-Worker situa-se entre 15% e 30% em latência e 20% a 40% em tokens, atribuíveis às chamadas de ida e volta (*dados de fornecedor único — não consolidados por estudo independente*) (MICROSOFT, 2026). Em contrapartida, a compressão de contexto por sub-agentes reduz o consumo de tokens do agente pai em até 90% (EPSILLA, 2026). Para tarefas de longa duração, esse ganho tende a superar o overhead de orquestração, tornando o sistema multi-agente mais barato que a alternativa mono-agente com contexto inflado. Recomenda-se difficulty-aware routing: modelos menores para roteamento e extração; modelos avançados para planejamento e consolidação (GROKIPEDIA, 2026). A Tabela 4 ilustra o efeito combinado das decisões de arquitetura.

Tabela 4. Efeito qualitativo de decisões de arquitetura sobre custo e latência

Decisão de arquitetura	Efeito em custo	Efeito em latência	Fonte
Sub-agentes com compressão de contexto	Reduz	Neutro/Positivo	Epsilla (2026)
Orchestrator-Worker	Aumenta	Aumenta	Microsoft (2026)
Parallel Fan-Out	Aumenta	Reduz	Digital Applied (2026); Pucek (2026)
Difficulty-aware routing	Reduz	Neutro	Grokikipedia (2026)
Checkpointing/pruning a 70%	Reduz	Neutro	Stackviv (2026)

Fonte: elaborado pelo autor. Nota: valores qualitativos baseados em fontes primárias dos respectivos fornecedores; não constituem meta-análise independente.

Recomenda-se aplicar compactação quando o contexto acumulado excede 70% do limite do modelo (STACKVIV, 2026).

8 Diretrizes de Engenharia para Produção

8.1 Gerenciamento Dinâmico de Contexto

Implementar sumarização incremental e pruning de histórico — remoção de metadados irrelevantes e respostas intermediárias. Aplicar compactação quando o contexto excede 70% do limite do modelo (STACKVIV, 2026), preservando artefatos de decisão e descartando rastros de baixo valor.

8.2 Otimização de Custos e Latência

Adotar difficulty-aware routing (GROKIPEDIA, 2026) e caching de contexto para reduzir custo marginal de requisições repetitivas.

8.3 Segurança e Isolamento

Sub-agentes devem operar sob o princípio do menor privilégio, restringindo acesso a APIs e bancos de dados ao escopo exato da tarefa. Implementar circuit breakers e cotas rígidas para prevenir loops infinitos (WITNESSAI, 2026). O isolamento de contexto — garantido pelo Claude Agent SDK (ANTHROPIC, 2026b) — limita o raio de explosão de um comprometimento.

8.4 Human-in-the-Loop

Observadores em group chat, revisores em ciclos maker-checker e escalção em handoff. Pontos de verificação obrigatórios tornam a orquestração síncrona, com estado persistido em checkpoints para retomada sem replay (MICROSOFT LEARN, 2026). A presença humana em pontos críticos de decisão é recomendada para domínios de alto risco.

8.5 Observabilidade

Stack mínimo: LangSmith (rastreamento de agentes), OpenTelemetry (métricas de infraestrutura) e dashboards de workflow engine, instrumentando toda operação e handoff (SHAKUDO, 2026; APISCOUT, 2026). Sem observabilidade, a depuração de topologias dinâmicas torna-se inviável dado o não determinismo das decisões de roteamento.

9 Riscos, Limitações e Estratégias de Mitigação

Não determinismo de roteamento. Decisões de LLMs em runtime podem divergir entre execuções equivalentes. Mitigação: abordagem híbrida com estrutura predefinida e limites de transição (AWS, 2025), persistindo traços completos para auditoria (SHAKUDO, 2026; APISCOUT, 2026). **Custo de tokens da orquestração.** Overhead de 20–40% em tokens do Orchestrator-Worker pode inviabilizar alto volume (*dados de fornecedor único*). Mitigação: difficulty-aware routing (GROKIPEDIA, 2026) e compressão de contexto por sub-agentes (EPSILLA, 2026). **Depuração de topologias dinâmicas.** Multiplicidade de caminhos dificulta causa raiz de falhas. Mitigação: observabilidade integral (SHAKUDO, 2026; APISCOUT, 2026) e time-travel debugging com checkpointing (AGENT MARKET CAP, 2026). **Segurança e loops.** Privilégios excessivos e ausência de circuit breakers induzem loops infinitos ou exfiltração de dados. Mitigação: princípio do menor privilégio e cotas rígidas por agente (WITNESSAI, 2026). **Interoperabilidade.** Fragmentação de protocolos (A2A, MCP) entre frameworks limita portabilidade. Mitigação: adoção de frameworks com suporte nativo aos protocolos padronizados (TZOLOV, 2026; MICROSOFT, 2026).

10 Tendências Emergentes

Seis tendências moldam o futuro dos dynamic workflows (ZYLOS RESEARCH, 2026; MICROSOFT LEARN, 2026): (1) **DAGs** como linguagem de especificação, com LangGraph como expoente; (2) **orquestração event-driven**, com agentes reagindo a eventos em barramentos (Kafka, RabbitMQ); (3) **modelo Actor**, cada agente como ator com caixa postal, popularizado por AutoGen e MAF; (4) **roteamento dinâmico por dificuldade**, com classificadores estimando complexidade da query para alocar recursos proporcionalmente; (5) **orquestração federada** cloud-edge através de fronteiras de confiança, com A2A como bloco fundamental; (6) **workflows auto-otimizáveis**, com meta-agentes ajustando a topologia com base em feedback de execuções anteriores. Essas tendências convergem dois movimentos: a formalização de grafos como linguagem comum de especificação (itens 1–3) e a autonomização progressiva da topologia (itens 4–6). A adoção antecipada de protocolos de interoperabilidade (A2A, MCP) é condição para capturar os benefícios da orquestração federada e auto-otimizável sem lock-in de fornecedor.

11 Roadmap de Adoção em Produção

Fase 1 — Fundação (semanas 1–4): estabelecer stack de observabilidade (LangSmith, OpenTelemetry) e modelo de isolamento de contexto e privilégios (WITNESSAI, 2026; SHAKUDO, 2026), iniciando com agentes generalistas simples. **Fase 2 — Orquestração Híbrida (semanas 5–10):** introduzir Orchestrator-Worker ou Sequential Pipeline conforme a tarefa, com compressão de contexto por sub-agentes (EPSILLA, 2026), difficulty-aware routing (GROKIPEDIA, 2026) e checkpoints para retomada sem replay (MICROSOFT LEARN, 2026). **Fase 3 — Escala e Autonomia (semanas 11+):** expandir para Parallel Fan-Out/Fan-In e Dynamic Handoff conforme necessidade de análise multi-perspectiva, habilitar human-in-the-loop, considerar Magentic para domínios de escopo aberto (ZYLOS RESEARCH, 2026), reavaliando topologia à luz de métricas de custo e latência. A recomendação central é evoluir de topologias estáticas para dinâmicas de forma incremental, sustentada por observabilidade desde o primeiro dia e arquitetura em camadas combinando durabilidade (Temporal), raciocínio de agente (LangGraph) e coordenação inter-serviço (Kafka + A2A).

12 Conclusão

Os padrões de orquestração multi-agente consolidaram-se como pilares de arquitetura em produção empresarial. A escolha do padrão e framework deve ser guiada por requisitos pragmáticos de latência, custos e complexidade da tarefa, e não por preferências de implementação. A compressão de contexto por sub-agentes é o principal benefício prático, com reduções de até 90% no consumo de tokens do agente pai em configurações reportadas (*dado de fornecedor único — não verificado independentemente*) (EPSILLA, 2026). Recomenda-se iniciar com agentes generalistas simples e aplicar especialização em sub-agentes apenas quando houver evidências mensuráveis de ganhos de performance ou conformidade. A arquitetura híbrida em camadas — durabilidade (Temporal), raciocínio de agente (LangGraph), coordenação inter-serviço (Kafka + A2A) e observabilidade desde o primeiro dia — evoluindo de topologias estáticas para dinâmicas de forma incremental, constitui a abordagem recomendada. Limitações atuais incluem depuração de topologias dinâmicas, custo de tokens da orquestração e fragmentação de protocolos entre frameworks. Perspectivas futuras apontam para DAGs como linguagem de especificação, orquestração event-driven e workflows auto-otimizáveis mediados por meta-agentes, cujo sucesso dependerá da disciplina de engenharia aplicada à observabilidade, segurança e human-in-the-loop.

Agradecimentos

[A preencher pelo autor.]

Conflito de Interesses

Os autores declaram não haver conflitos de interesses financeiros, comerciais ou acadêmicos relacionados ao presente trabalho.

Referências

AGENT MARKET CAP. LangGraph vs AutoGen vs CrewAI vs DSPy: The 2026 Multi-Agent Framework Decision Guide. **Agent Market Cap**, 2026. Disponível em: <https://agentmarketcap.ai/blog/2026/04/11/langgraph-autogen-crewai-dspy-multi-agent-orchestration-2026>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Agent SDK**: Subagents in the SDK. **Anthropic Documentation**, 2026a. Disponível em: <https://code.claude.com/docs/en/agent-sdk/subagents>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Code**: Orchestrate subagents at scale with dynamic workflows. **Anthropic Documentation**, 2026b. Disponível em: <https://code.claude.com/docs/en/workflows>. Acesso em: 12 jul. 2026.

APISCOUT. **OpenAI Agents SDK**: Architecture Patterns 2026. **APIScout**, 2026. Disponível em: <https://apiscout.dev/guides/openai-agents-sdk-architecture-patterns-2026>. Acesso em: 12 jul. 2026.

AWS. **Agentic AI patterns and workflows on AWS**. Amazon Web Services, 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/introduction.html>. Acesso em: 12 jul. 2026.

DEVARAJAN, Vishalini. **Common Workflow Patterns for AI Agents**. **GUVI Blog**, 2026. Disponível em: <https://www.guvi.in/blog/common-workflow-patterns-for-ai-agents>. Acesso em: 12 jul. 2026.

DIGITAL APPLIED. **Multi-Agent Orchestration**: 5 Patterns That Work in 2026. **Digital Applied**, 2026. Disponível em: <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work>. Acesso em: 12 jul. 2026.

EPSILLA. **The 3 Essential Sub-Agent Patterns for Production-Grade AI Systems**. **Epsilla Blog**, 2026. Disponível em: <https://www.epsilla.com/blogs/2026-03-14-ai-sub-agent-patterns>. Acesso em: 12 jul. 2026.

GARTNER. Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025. **Gartner Newsroom**, 26 ago. 2025. Disponível em: <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>. Acesso em: 12 jul. 2026.

GHEWARE, Rajesh. **LangGraph vs CrewAI vs AutoGen**: Complete AI Agent Framework Comparison 2026. **DevOps Gheware**, 2026. Disponível em: <https://devops.gheware.com/blog/posts/langgraph-vs-crewai-vs-autogen-comparison-2026.html>. Acesso em: 12 jul. 2026.

GROKIPEDIA. **AI Agent Orchestration**. **Grokikipedia**, 2026. Disponível em: https://grokipedia.com/page/AI_Agent_Orchestration. Acesso em: 12 jul. 2026.

JETTHOUGHTS. **LangGraph vs CrewAI vs AutoGen**: Open Source Alternatives 2025. **JetThoughts**, 2026. Disponível em: <https://jetthoughts.com/blog/autogen-crewai-langgraph-ai-agent-frameworks-2025>. Acesso em: 12 jul. 2026.

LUSHBINARY. **Multi-Agent AI Orchestration Patterns**: Supervisor, Swarm, Pipeline & Router. **Lushbinary**, 2026. Disponível em: <https://lushbinary.com/blog/multi-agent-orchestration-patterns-supervisor-swarm-pipeline-router-guide>. Acesso em: 12 jul. 2026.

META INTELLIGENCE. **LangGraph vs CrewAI vs AutoGen**: AI Agent Framework Comparison [2026]. **Meta Intelligence**, 2025. Disponível em: <https://www.meta-intelligence.tech/en/insight-ai-agent-frameworks>. Acesso em: 12 jul. 2026.

MICROSOFT. **AI Agent Orchestration Patterns**. Microsoft Learn, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>. Acesso em: 12 jul. 2026.

MICROSOFT LEARN. **Multi-agent patterns**. Microsoft Learn, 2026. Disponível em: <https://learn.microsoft.com/en-us/agents/architecture/multi-agent-patterns>. Acesso em: 12 jul. 2026.

OPENAI. **Agents SDK: Orchestration and handoffs**. OpenAI Documentation, 2026. Disponível em: <https://developers.openai.com/api/docs/guides/agents/orchestration>. Acesso em: 12 jul. 2026.

OPENAGENTS. **CrewAI vs LangGraph vs AutoGen vs OpenAgents: Best AI Agent Framework**. OpenAgents, 2026. Disponível em: <https://openagents.org/blog/posts/2026-02-23-open-source-ai-agent-frameworks-compared>. Acesso em: 12 jul. 2026.

PUCEK, Bartek. **Orchestration Patterns for AI Agent Workflows**. The Thinking Company, 2026. Disponível em: <https://thinking.inc/en/blue-ocean/agentic/agent-orchestration-patterns>. Acesso em: 12 jul. 2026.

ROCKB. **Microsoft Agent Framework 2026: AutoGen Successor Explained**. RockB, 2026. Disponível em: <https://baeseokjae.github.io/posts/microsoft-agent-framework-2026/>. Acesso em: 12 jul. 2026.

SHAKUDO. **Enterprise Agentic AI Workflow Patterns for 2025**. Shakudo, 2025. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/69388ca4cdb5836ee83b10f5_69388ca257d8a9675e92aeb8_agentic-ai-workflow-patterns-whitepaper.pdf. Acesso em: 12 jul. 2026.

SHAKUDO. **Multi-Agent Systems Transform Enterprise AI in 2026**. Shakudo, 2026. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/696fdd9dcee4a9c14251480b_696fdd9c3876035a277d2a6f_multi-agent-systems-2026-trends-whitepaper.pdf. Acesso em: 12 jul. 2026.

STACKVIV. **Agentic AI & Multi-Agent Systems: Advanced Guide**. Stackviv, 2026. Disponível em: <https://stackviv.ai/blog/agentic-ai-multi-agent-systems-guide>. Acesso em: 12 jul. 2026.

SUBAGENT.DEV. **Orchestration Patterns**. SubAgent.Dev, 2026. Disponível em: <https://subagent.developersdigest.tech/patterns>. Acesso em: 12 jul. 2026.

TZOLOV, Christian. **Spring AI Agentic Patterns (Part 4): Subagent Orchestration**. Spring Blog, 2026. Disponível em: <https://spring.io/blog/2026/01/27/spring-ai-agentic-patterns-4-task-subagents/>. Acesso em: 12 jul. 2026.

VELLUM AI. **The 2026 Guide to AI Agent Workflows**. Vellum AI, 2025. Disponível em: <https://www.vellum.ai/blog/agentic-workflows-emerging-architectures-and-design-patterns>. Acesso em: 12 jul. 2026.

WINBUZZER. **Anthropic Shows How to Scale Claude Code with Subagents and MCP**. WinBuzzer, 2026. Disponível em: <https://winbuzzer.com/2026/03/24/anthropic-claude-code-subagent-mcp-advanced-patterns-xcxbn>. Acesso em: 12 jul. 2026.

WITNESSAI. **AI Agent Orchestration: Managing Multi-Agent Workflows**. WitnessAI, 2026. Disponível em: <https://witness.ai/blog/ai-agent-orchestration>. Acesso em: 12 jul. 2026.

ZYLOS RESEARCH. **Agent Workflow Orchestration Patterns: DAG, Event-Driven, and Actor Models**. Zylos Research, 2026. Disponível em: <https://zylos.ai/research/2026-04-14-agent-workflow-orchestration-patterns>. Acesso em: 12 jul. 2026.